Warehouse Management with Reinforcement Learning

Elias Al Bouzidi, Massine El Khader, and Idriss Benkirane

Reinforcement Learning Course Project

Abstract. We study warehouse control in two reinforcement learning environments. The first environment is a discrete seasonal inventory problem with three warehouses organized around one central hub. The second environment is a continuous transport problem with four warehouses organized as one hub and three spokes, where the policy directly outputs shipment quantities on each route. We chose this problem because it combines uncertain demand, operational constraints, and a clear economic structure where good decisions must balance service quality and transport cost. In the discrete setting we compare random control, a hand built heuristic, value iteration, Q Learning, SARSA, and DQN. In the continuous setting we use PPO with a Gaussian policy. The main result is that reinforcement learning learns policies that are both effective and interpretable. In the discrete environment DQN reaches the best average reward at -732.65 while value iteration achieves the best service level at 72.1%. In the continuous environment PPO improves reward from about -16000 to about -7000 and reduces transport cost from about 800 to about 10. The learned policies recover the intended routing logic, import or move stock through the hub and send inventory to the spoke warehouses when needed. Code is available at `https://github.com/MassineELKHADER/Warehouse-management-RL-Course`.

Keywords: Reinforcement Learning · Inventory Control · Warehouse Management

1 Introduction

Our project focuses on a hub and spoke logistics structure. One warehouse acts as a central hub with cheaper supply access and larger capacity, while the other warehouses have tighter limits and more expensive direct replenishment. This creates a clear planning question. Should the controller import locally at each warehouse, or should it concentrate stock at the hub and redistribute it?

We chose this problem for three reasons. First, decisions are repeated, demand is uncertain, and bad actions create immediate operational costs. Second, it contains a meaningful trade off between service level and cost. Third, it lets us study both a structured discrete action space and a more flexible continuous action space. The discrete environment is useful for planning and tabular methods. The continuous environment tests whether policy gradient methods can learn shipment magnitudes directly.

2 Environments

2.1 Discrete environment

The discrete environment models three warehouses. Warehouse $W1$ is the hub with capacity 30, import cost 1.0, and holding cost 0.3. Warehouses $W2$ and $W3$ are spokes with capacity 15, import cost 5.0, and holding cost 1.0. Transfers from the hub to a spoke cost 0.5 per unit, transfers from a spoke back to the hub cost 1.5, and transfers between spokes cost 3.0.

The state is

$$s_t = (\text{stock}_1, \text{stock}_2, \text{stock}_3, \text{month})$$

where the month index captures seasonality over a horizon of 12 steps. Demand at warehouse i in month m follows a Poisson model

$$D_i(m) \sim \text{Poisson}(\mu_i(m))$$

with seasonal mean

$$\mu_i(m) = \max\left(1, \text{base}_i + \text{amplitude}_i \sin\left(\frac{2\pi(m - \text{phase}_i)}{12}\right)\right).$$

The episode starts from inventory $(15, 7, 7)$. Figure 1 summarizes both the network structure and the seasonal demand pattern that define this environment.

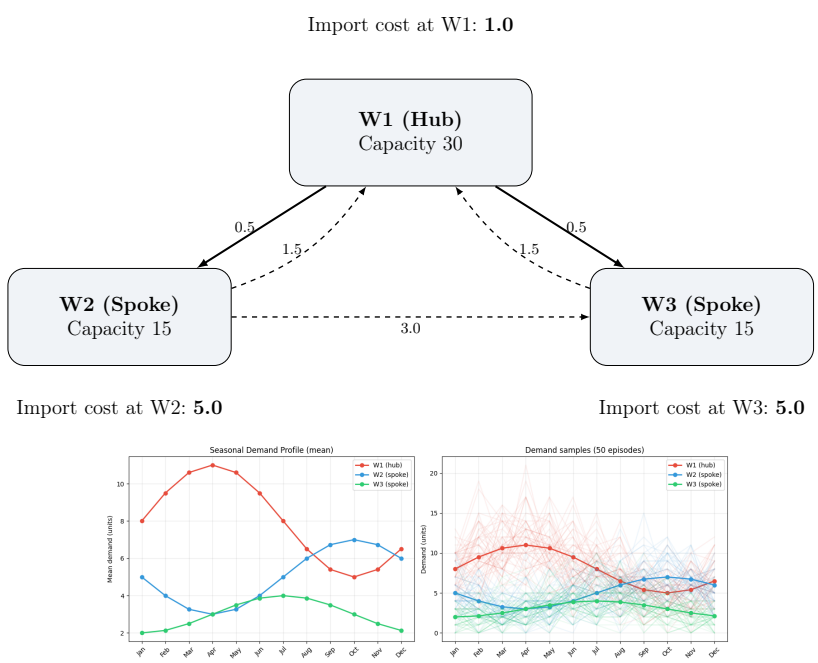

Fig. 1. Discrete environment structure. Top, the hub and spoke network, where edge weights represent transfer cost per unit. Bottom, the seasonal demand profile.

The discrete controller uses 34 actions. Each action combines an import vector $[i_1, i_2, i_3]$ with an optional list of transfers $(\text{src}, \text{dst}, \text{qty})$. The action set includes different import quantities (2, 4, or 6 units into one warehouse), direct transfers (moving stock between warehouses), bulk imports (importing into all warehouses at once), and hub centered combined actions (importing at $W1$ and redistributing to one or both spokes in the same step).

Action type	Count
No-op	1
Single imports	9
Transfer-only	12
Bulk imports	2
Hub import + transfer	6
Hub-specific large actions	4

At each step the environment applies imports, applies transfers, samples demand, fulfills demand, updates inventory, and computes the reward

$$\begin{aligned} R_t = & -10 \sum_i \text{unmet}_i - \sum_i c_i^{\text{import}} \text{import}_i \\ & - \sum_{i,j} c_{ij}^{\text{transfer}} \text{transfer}_{ij} - \sum_i c_i^{\text{hold}} \text{stock}'_i - \text{infeasible units}. \end{aligned} \tag{1}$$

This reward penalizes unmet demand most strongly, then import and transfer costs, then leftover stock, while also penalizing infeasible action components that exceed capacity or available inventory.

2.2 Continuous environment

The continuous environment extends the control problem to direct shipment planning. We use $N = 4$ warehouses organized as one hub and three spokes. The state is a dictionary containing current inventory and current demand vectors, so the flattened observation has dimension $2N$. The action is a transport matrix $T \in \mathbb{R}^{N \times N}$ where T_{ij} is the quantity shipped from warehouse i to warehouse j. Raw actions are clipped to non negative values, self shipments are forced to zero, and each row is rescaled when the outgoing quantity exceeds available inventory. This guarantees feasibility at each step.

For the continuous experiment the cost matrix follows the same hub and spoke idea as the discrete problem. Routes through the hub are cheaper than spoke to spoke moves. The reward is

$$R_t = -C_{\text{transport}} - \lambda_d \sum_i U_i$$

with $C_{\text{transport}} = \sum_{i,j} c_{ij} T_{ij}$ and $U_i = \max(0, D_i - I_i^{\text{post}})$ the unmet demand after shipping. This setting removes the hand designed discrete action list and lets the agent choose shipment magnitudes directly.

3 Agents

In the discrete environment we compare six controllers. The random baseline samples one of the 34 actions uniformly. The heuristic controller follows the intended logistics logic, keep the hub stocked, send stock from the hub to low spokes, and avoid expensive spoke imports when possible. Value iteration solves a finite horizon planning problem on a coarsened state space with stock step size 4.

We also train three reinforcement learning agents. Q Learning updates action values with off policy temporal difference learning [1]. SARSA uses the same coarsened state representation and hyperparameters but performs on policy updates [1]. DQN approximates $Q(s, a)$ with a neural network that takes normalized stocks and seasonal month features as input and outputs the value of the 34 actions [1]. The network uses 6 inputs, two hidden layers of size 64.

For the continuous environment we use PPO [2]. The policy is Gaussian over the flattened transport matrix. We use a network with hidden sizes $[128, 128]$, learning rate 3×10^{-4}, discount factor 0.99, and GAE parameter 0.95.

4 Results and discussion

4.1 Discrete environment

Table 1 and Fig. 2 summarize the main results after training. All learning based methods are much stronger than random control. The heuristic baseline is already useful because it encodes the hub first idea. Value iteration gives the best service level and the lowest unmet demand. DQN gives the best average reward and nearly matches value iteration on service quality. SARSA is more stable than Q Learning and finishes with better reward and fewer infeasible actions.

Table 1. Discrete environment results averaged over 200 evaluation episodes.

Method	Avg Reward	Std	Service (%)	Unmet	Imports	Transfers	Infeas.
Random	-1416.75	155.05	34.0	127.25	37.0	13.2	12.6
Heuristic	-977.21	109.25	55.2	86.28	79.4	14.5	0.0
Value Iteration	-735.44	108.16	**72.1**	54.04	111.8	29.2	0.1
Q Learning	-856.50	104.16	64.4	68.86	96.9	20.0	0.3
SARSA	-841.07	108.43	66.7	64.34	101.7	16.6	0.0
DQN	**-732.65**	109.84	71.2	55.85	109.9	31.6	0.2

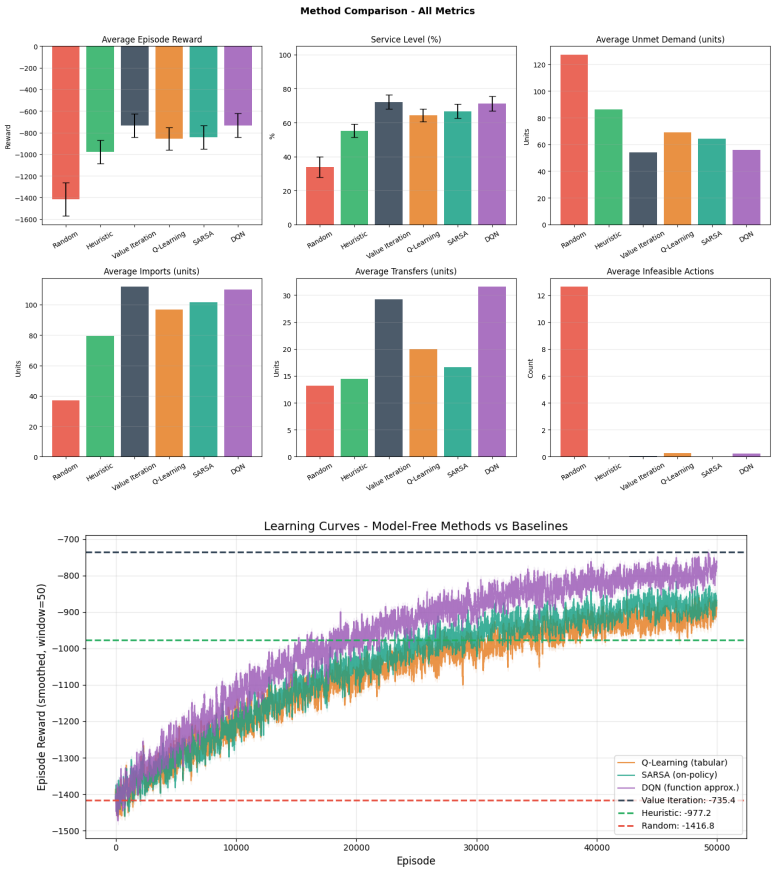

Fig. 2. Top, comparison across methods. Bottom, learning curves for Q Learning, SARSA, and DQN.

The most important qualitative result is policy structure. The best policies learn the intended hub centered operating logic rather than relying on direct spoke imports. For DQN, average imports are about 109.7 at the hub and only about 0.1 at each spoke, while value iteration shows the same pattern with about 106.7 hub imports and only about 2.5 at each spoke.

4.2 Continuous environment

The continuous setting is harder because the policy must output a full transport matrix instead of choosing from a small action list. PPO still learns a meaningful strategy. During training the episode reward improves from about -16000 to about -7000. At the same time transport cost falls from about 800 to about 10. Final demand satisfaction reaches about 78%, which is close to the greedy baseline while using much lower transport cost.

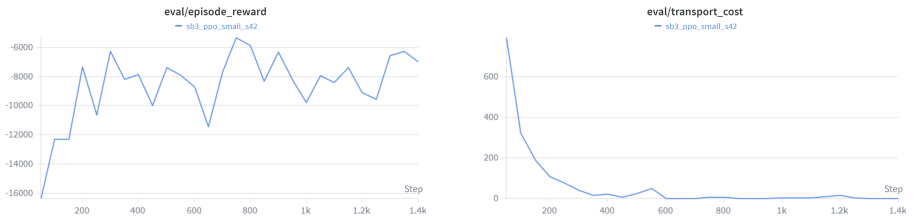

Fig. 3. Continuous environment training curves for reward and transport cost under PPO.

These results show both the strength and the limitation of the continuous setup. The agent clearly learns to reduce unnecessary movement and to focus on cheaper routes, but the remaining service gap suggests that continuous control under stochastic demand is still difficult. At the same time, the continuous setting remains less solved than the discrete one, which suggests that stochastic demand and continuous routing decisions make learning more difficult.

5 Conclusion

This project studied warehouse management in two settings with the same operational idea, centralize inventory at a hub and redistribute it efficiently to the spokes. In the discrete seasonal environment, reinforcement learning clearly improves over random control and over a simple hand designed policy. The strongest methods are value iteration and DQN. Value iteration achieves the best service level at 72.1%, while DQN achieves the best average reward at -732.65. In addition to strong quantitative performance, both methods recover the intended hub and spoke behavior, with almost all imports concentrated at $W1$ and the spoke warehouses mainly supplied by transfers.

In the continuous environment, PPO also learns a useful policy even though the problem is harder. Reward improves from about -16000 to about -7000, transport cost drops from about 800 to about 10, and final demand satisfaction reaches about 78%. This result is encouraging because the agent must choose shipment magnitudes directly in a higher dimensional action space. At the same time, the continuous setting remains less solved than the discrete one, which suggests that stochastic demand and continuous routing decisions make learning more difficult.

References

1. Christodoulidis, S.: Reinforcement Learning course notes. CentraleSupélec, Université Paris-Saclay (2026)
2. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. 2nd edition. MIT Press, Cambridge, MA (2018)